

Experiment Name

Write a program to identify unique identifiers from a C program using a basic lexical analyzer.

Objective

The objective of this experiment is to design and implement a basic lexical analyzer that scans a C program and detects valid identifiers. The program identifies unique identifiers while ignoring keywords, invalid tokens, and duplicates. This helps in understanding the working of lexical analysis in compiler design.

Algorithm

1. Start the program.
2. Read the input C program from a file or user input.
3. Define a list of C keywords (such as int, float, char, if, while, return, etc.).
4. Initialize an empty set or list to store unique identifiers.
5. Read the program line by line.
6. For each line, break the line into tokens using delimiters such as spaces, commas, semicolons, parentheses, and operators.
7. For each token:
 - (a) Check if the token starts with a letter (a–z or A–Z) or an underscore (_).
 - (b) Check if the remaining characters of the token contain only letters, digits, or underscores.
 - (c) If the token does not satisfy identifier rules, skip it.
 - (d) Compare the token with the list of C keywords.
 - (e) If the token matches a keyword, ignore it.
 - (f) Otherwise, check whether the identifier is already stored.
 - (g) If it is not present, add it to the list of unique identifiers.
8. Continue scanning until the entire program is processed.
9. Display all unique identifiers found in the C program.
10. Stop the program.

Source Code

```
1  #include <bits/stdc++.h>
2  using namespace std;
3  bool isKeyword(const string &word)
4  {
5      static const vector<string> keywords = {
6
7          "auto", "break", "case", "char", "const", "continue", "default", "do", "double",
8
9          "else", "enum", "extern", "float", "for", "goto", "if", "int", "long", "register",
10         "return", "short", "signed", "sizeof", "static", "struct", "switch", "typedef",
11         "union", "unsigned", "void", "volatile", "while"
12     };
13
14     for (const auto &k : keywords)
15         if (word == k)
16             return true;
17
18     return false;
19 }
20
21 bool isIgnored(const string &word)
22 {
23     static const vector<string> ignored = {
24         "include", "stdio", "iostream", "printf", "scanf",
25         "cin", "cout", "endl"
26     };
27
28     for (const auto &w : ignored)
29         if (word == w)
30             return true;
31
32     return false;
33 }
34
35 bool isValidIdentifier(const string &word)
36 {
37     if (word.empty() || !(isalpha(word[0]) || word[0] == '_'))
38         return false;
39
40     for (char c : word)
41         if (!(isalnum(c) || c == '_'))
42             return false;
43
44     return true;
45 }
46
47 int main()
48 {
49     ifstream file("input.c");
50     map<string, int> frequency;
51     string word;
52
53     while (file >> word)
54     {
55         word = cleanToken(word);
56
57         if (isValidIdentifier(word) &&
58             !isKeyword(word) &&
59             !isIgnored(word))
60         {
61             frequency[word]++;
62         }
63     }
64
65     for (const auto &entry : frequency)
66     {
67         cout << left << setw(20)
68              << entry.first
69              << entry.second << "\n";
70     }
71
72     return 0;
73 }
```

Figure 1: Source code for lexical analyzer to detect unique identifiers

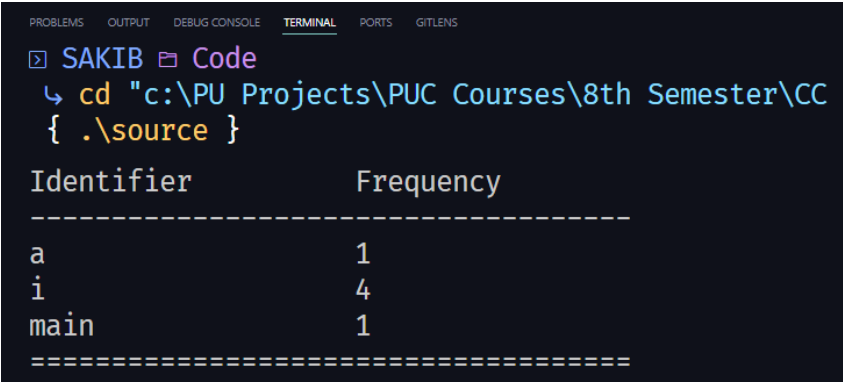
```

1  #include <stdio.h>
2
3  int main()
4  {
5      int i;
6      int a[10];
7      for(i = 1; i <= 10; i++)
8      {
9          printf("Bangladesh\n");
10     }
11
12     return 0;
13 }

```

Figure 2: Input code for lexical analyzer to detect unique identifiers

Output Screenshot



```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  GITLENS
SAKIB  Code
cd "c:\PU Projects\PUC Courses\8th Semester\CC"
{ .\source }

Identifier      Frequency
-----
a               1
i               4
main            1
=====

```

Figure 3: Output showing all unique identifiers extracted from the C program

Discussion

This experiment demonstrates the design of a simple lexical analyzer to extract identifiers from a C program. The analyzer scans the input, applies identifier rules, removes keywords, and ensures uniqueness using a suitable data structure. This experiment provides a clear understanding of tokenization, identifier recognition, and the importance of lexical analysis in the first phase of a compiler.



Department of Computer Science and Engineering
Premier University

CSE 454: Compiler Construction Laboratory

Lab Report 06: Write a program to detect and remove left recursion
from a given context-free grammar using C++

Submitted by:

Name	Mohammad Hafizur Rahman Sakib
ID	0222210005101118
Section	C
Session	Fall 2025 (8th Semester)
Date of performance	11.03.2026
Submission Date	31.03.2026

Submitted to:

Nazma Akther
Assistant Professor, Department of CSE
Premier University, Chattogram

Remarks

Experiment Name

Write a program to detect and remove left recursion from a given context-free grammar using C.

Objective

The objective of this experiment is to design and implement a C program that takes a context-free grammar as input, detects left recursion in the productions, and removes it by transforming the grammar into an equivalent non-left-recursive grammar. This helps in understanding the importance of eliminating left recursion before applying top-down parsing techniques in compiler design.

Algorithm

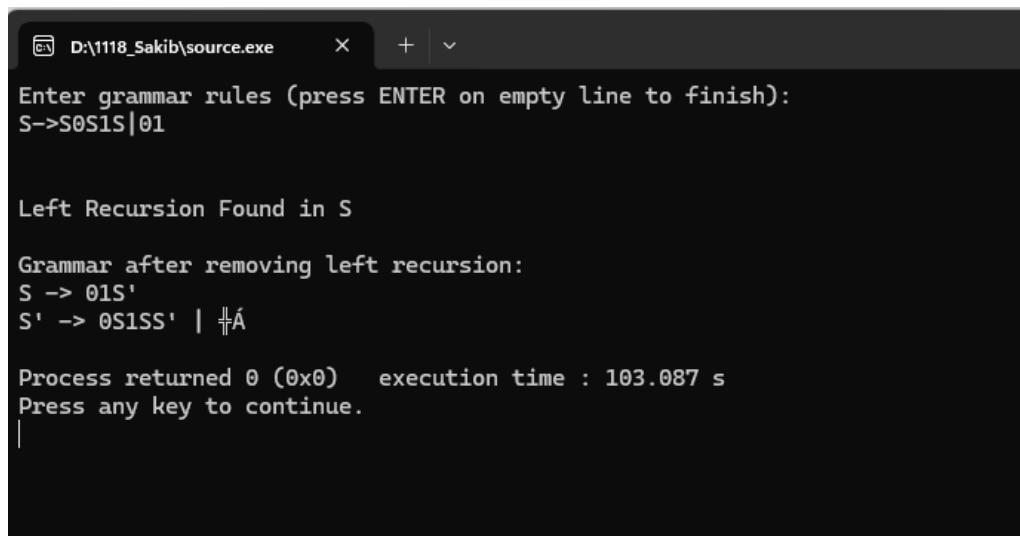
1. Start the program.
2. Read the number of grammar productions or read rules until an empty line.
3. For each production of the form $A \rightarrow \alpha$:
 - (a) Extract the left-hand side (non-terminal) and right-hand side.
 - (b) Split the right-hand side into individual productions using '—' as delimiter.
 - (c) Separate the productions into two groups:
 - Those starting with the same non-terminal A (left recursive - α part).
 - Those not starting with A (β part).
 - (d) If left recursive productions exist:
 - Print the original production and indicate left recursion is found.
 - Transform the grammar using the standard left recursion removal rule:
$$A \rightarrow \beta A' \mid \beta$$
$$A' \rightarrow \alpha A' \mid \epsilon$$
 - (e) If no left recursion is found, print the production as it is.
4. Continue until all productions are processed.
5. Stop the program.

Source Code

```
1  #include <stdio.h>
2  #include <string.h>
3
4  #define MAX 50
5  #define LEN 100
6
7  void removeLR(char g[MAX][LEN], int n) {
8      for (int i = 0; i < n; i++) {
9          char L[LEN], R[LEN], *p, prod[10][LEN], a[10][LEN], b[10][LEN];
10         int pc = 0, ac = 0, bc = 0;
11
12         sscanf(g[i], "%[^-]>%[^\\n]", L, R);
13
14         p = strtok(R, "|");
15         while (p) { strcpy(prod[pc++], p); p = strtok(NULL, "|"); }
16
17         for (int j = 0; j < pc; j++)
18             if (!strcmp(prod[j], L, strlen(L)))
19                 strcpy(a[ac++], prod[j] + strlen(L));
20             else
21                 strcpy(b[bc++], prod[j]);
22
23         if (ac) {
24             printf("\nLeft Recursion in %s\n", L);
25             for (int j = 0; j < bc; j++)
26                 printf("%s%s'", b[j], L, j < bc - 1 ? " | " : "");
27
28             printf("\n%s' -> ", L);
29             for (int j = 0; j < ac; j++)
30                 printf("%s%s'", a[j], L, j < ac - 1 ? " | " : "");
31
32             printf(" | ε\n");
33         } else {
34             printf("\nNo left recursion: %s\n", g[i]);
35         }
36     }
37 }
38
39 int main() {
40     char g[MAX][LEN];
41     int n = 0;
42
43     printf("Enter grammar (empty line to stop):\n");
44     while (fgets(g[n], LEN, stdin) && g[n][0] != '\n') {
45         g[n][strlen(g[n], "\n")] = 0;
46         n++;
47     }
48
49     removeLR(g, n);
50     return 0;
51 }
```

Figure 1: Source code for detecting and removing left recursion from a given grammar

Output Screenshot



```
D:\1118_Sakib\source.exe  X  +  v
Enter grammar rules (press ENTER on empty line to finish):
S->S0S1S|01

Left Recursion Found in S

Grammar after removing left recursion:
S -> 01S'
S' -> 0S1SS' | $Á

Process returned 0 (0x0)   execution time : 103.087 s
Press any key to continue.
|
```

Figure 2: Output showing detection and removal of left recursion

Discussion

This experiment demonstrates the implementation of left recursion removal, which is a crucial step in making a grammar suitable for predictive (top-down) parsing. The program correctly identifies left-recursive productions by checking whether any alternative on the right-hand side begins with the same non-terminal as the left-hand side. It then applies the standard transformation to eliminate direct left recursion while preserving the language generated by the grammar.

The use of string tokenization with `strtok()` and careful parsing of productions makes the implementation efficient for laboratory purposes. This experiment provides hands-on understanding of grammar transformation techniques used in the syntax analysis phase of a compiler.